



University of San Carlos | Department of
COMPUTER ENGINEERING

CpE 3104 - Microprocessors

I/O Address Decoding and Basic Interfacing

Lecture 6-1

Unit 6: INPUT/OUTPUT (I/O) INTERFACING

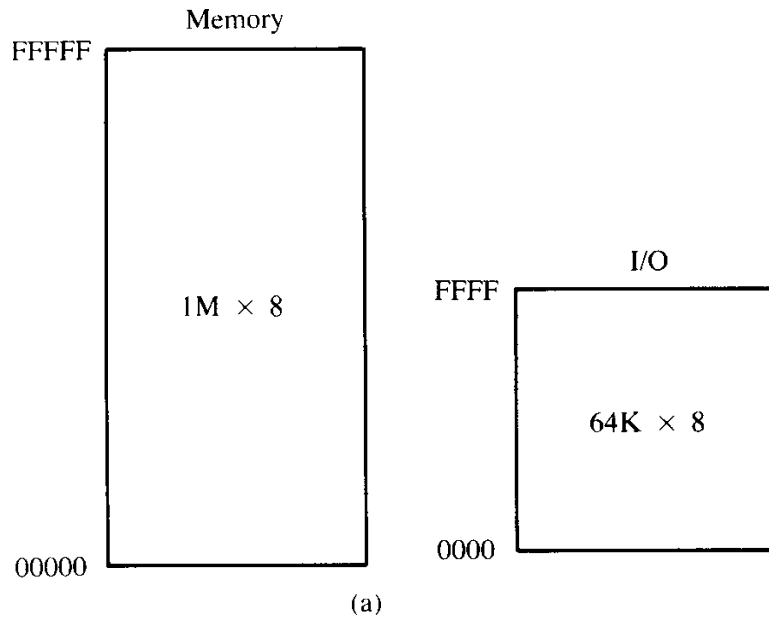
© 2025

Contents

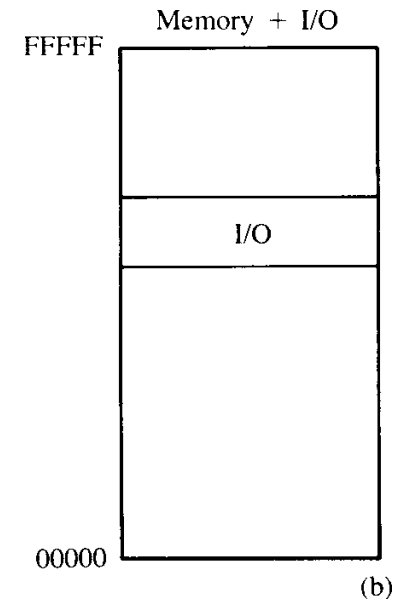
- I/O Interfacing Methods
- Basic I/O Interfaces
- I/O Port Address Decoding
- Introduction to 8255 Programmable Peripheral Interface

Methods of I/O Interfacing

- **Isolated I/O** - the I/O locations are separated from the memory system and it has its own address space.
- **Memory Mapped I/O** - I/O devices are treated as a memory location in the memory map.



Isolated I/O



Memory-Mapped I/O

I/O Instructions

- **Isolated I/O** - uses the IN, OUT, INS and OUTS where these instructions affects will make the $\overline{MIO}$ pin to logic-0 (signifying an I/O write or read)
- **Memory Mapped I/O** - does not use the above instructions, instead, it uses any instruction that transfers data between the microprocessor and memory.

I/O Instructions

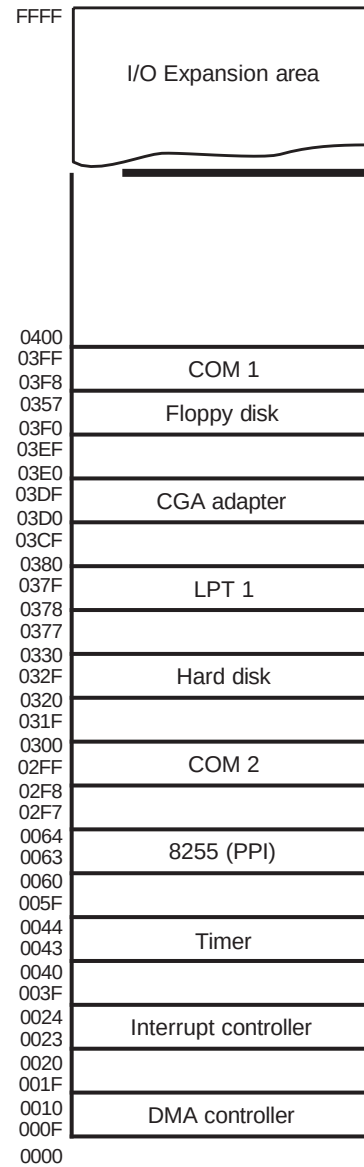
- **Isolated I/O:**

```
MOV DX, 0F0H ; move the I/O address to DX
MOV AL, 80H   ; move data to be sent to I/O to AL
OUT DX, AL    ; send data in AL to I/O address in DX
```

- **Memory Mapped I/O**

```
MOV AL, 80H ; move data to be sent to I/O to AL
MOV 1000H, AL ; send data in AL to I/O address DS:1000H
```

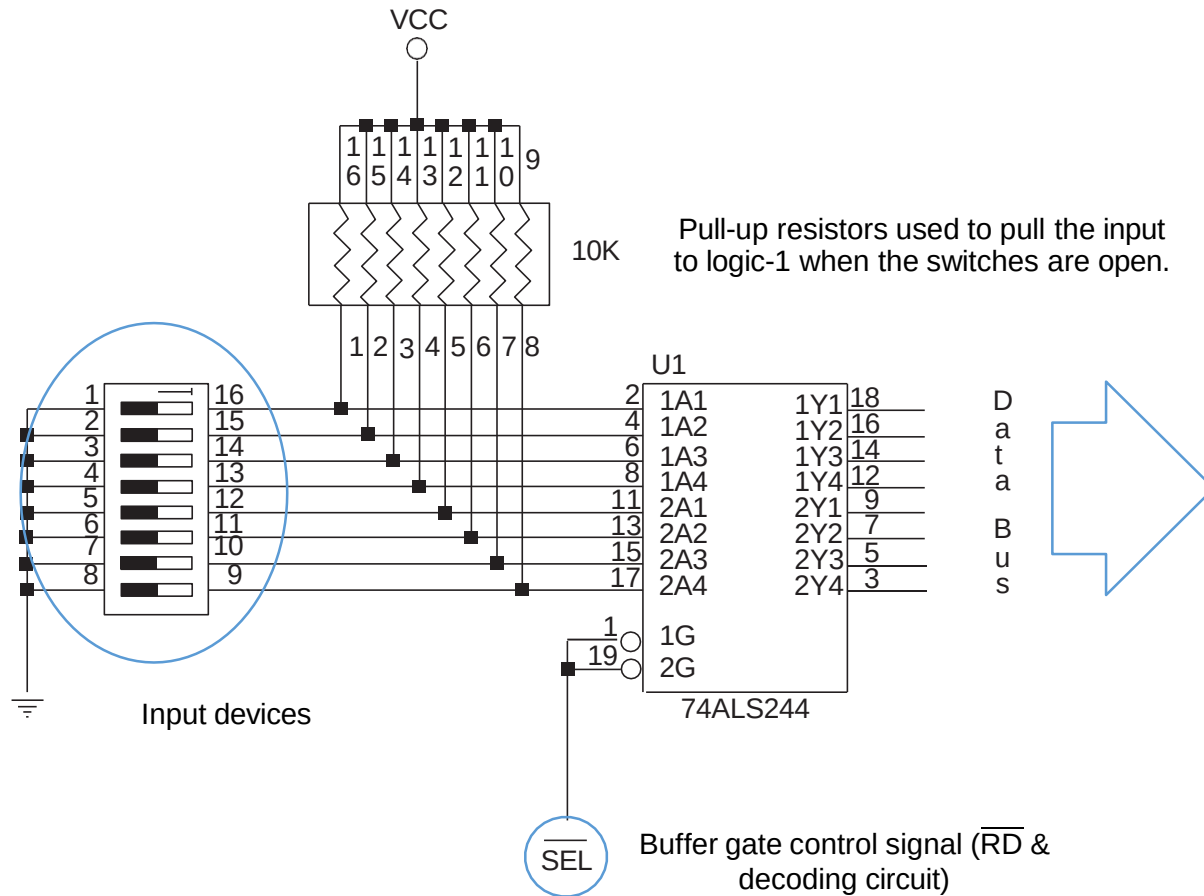
The I/O map of a personal computer illustrating many of the fixed I/O areas.



Basic I/O Interfaces

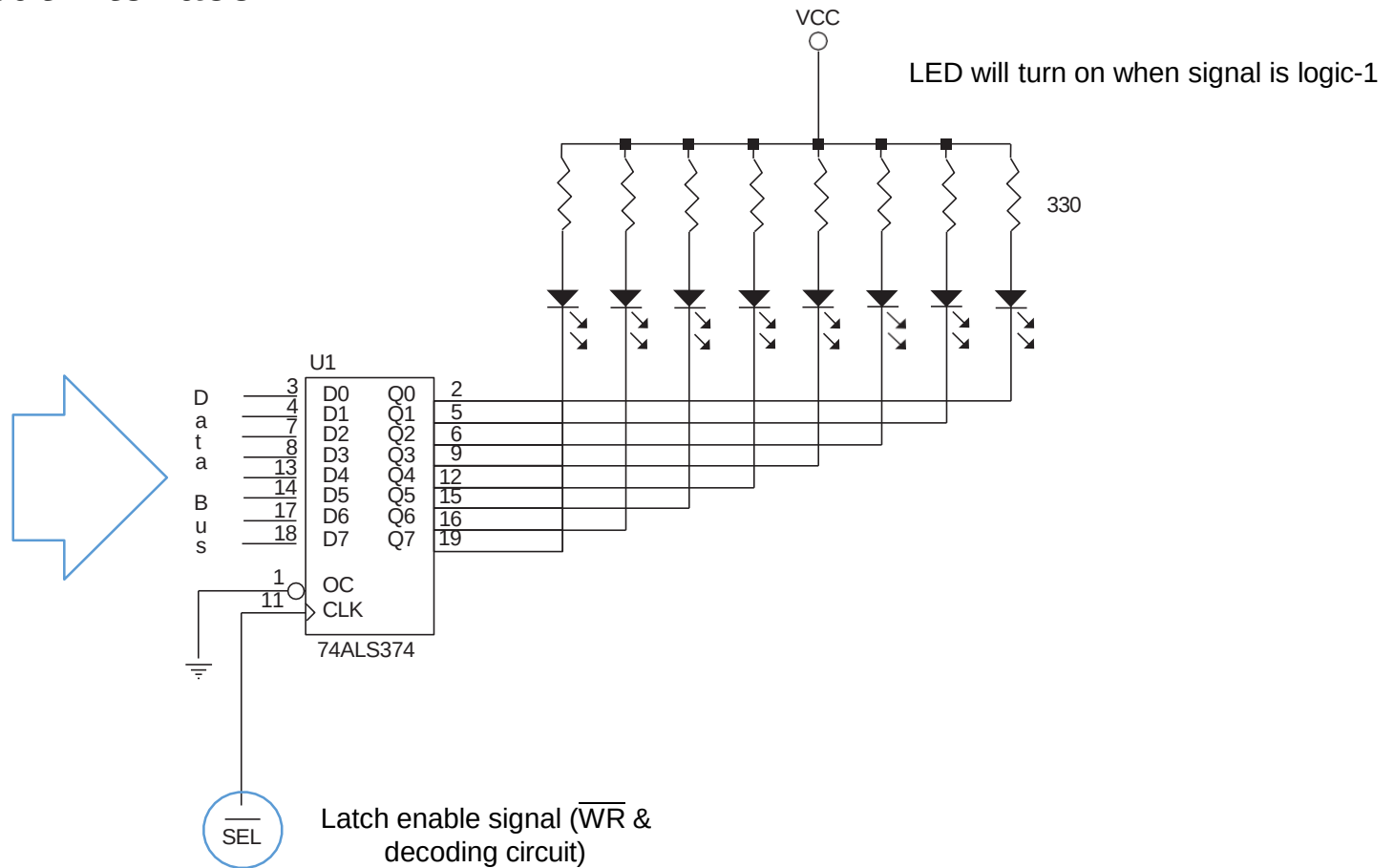
- The basic input device is a set of three-state **buffers** such as the 74LS244.
- The basic output device is a set of data **latches** such as the 74LS373.
- The term **IN** refers to moving data from the I/O device into the microprocessor and the term **OUT** refers to moving data out of the microprocessor to the I/O device.

Basic Input Interface



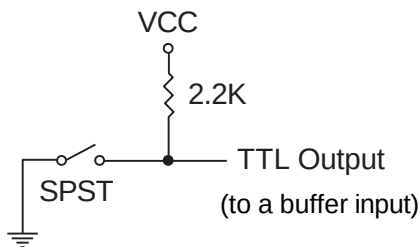
The basic input interface illustrating the connection of eight switches.
Note that the **74ALS244** is a three-state buffer that controls the application of the switch data to the data bus.

Basic Output Interface

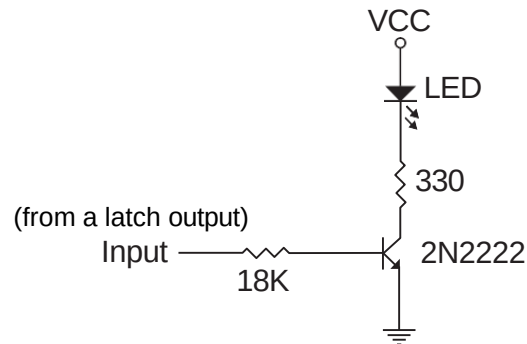


The basic output interface using a **74ALS374** connected to a set of LED displays.

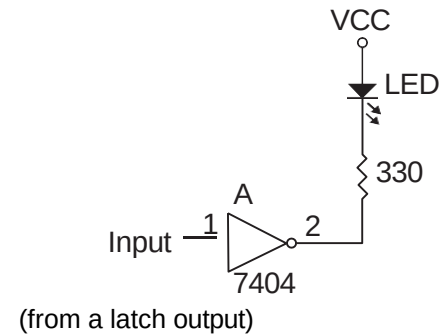
Proper I/O Circuit Biasing



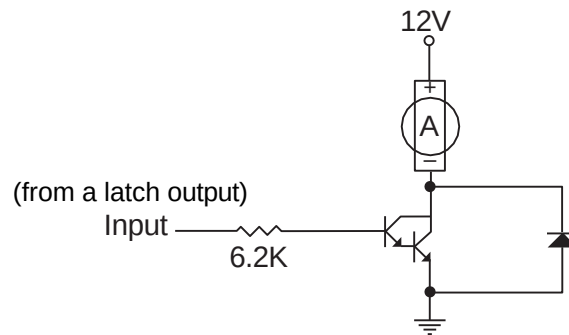
A single- pole, single-throw switch interfaced as a TTL device.



Interfacing an LED using a transistor.



Interfacing an LED using an inverter.



A DC motor interfaced to a system by using a Darlington-pair.

I/O Address Decoding

- I/O port address decoding is very similar to memory address decoding, especially for memory-mapped I/O devices.
- The main difference between memory decoding and isolated I/O decoding is the number of address pins connected to the decoder.
- A0 – A19 are decoded for memory and A0 – A15 are decoded for isolated I/O.
- Only 8-bit addresses will be discussed for isolated I/O.

8-bit I/O Address Decoding

- The fixed I/O instruction uses an 8-bit I/O port address that appears on $A_0 - A_{15}$ as 0000H–00FFH.
- If a system will never contain more than 256 I/O devices, we often decode only address connections $A_0 - A_7$ for an 8-bit I/O port address.
- For example: 00H - FFH (I/O address range)

Memory Address vs I/O Address

Memory Address: ¹⁹0000 ¹³00xx xxxx xxxx ⁰xxxx (00000H-03FFFH)

I/O Address: ¹⁹0000 0000 0000 ⁷xxxx ⁰xxxx (00H-FFH)

Example:

Memory Address: ¹⁹0000 0000 0000 ⁷1111 ⁰0000 **000F0H**

I/O Address: ¹⁹0000 0000 0000 ⁷1111 ⁰0000 **000F0H**

Same physical
address

I/O address needs to be decoded from A₀ - A₇ with **MIO** to enable only I/O and disable the memory (both Ram & ROM). This will solve the instance where the memory and I/O have the same physical address.

I/O Address Decoder

I/O Address Range: **F0H - F7H**

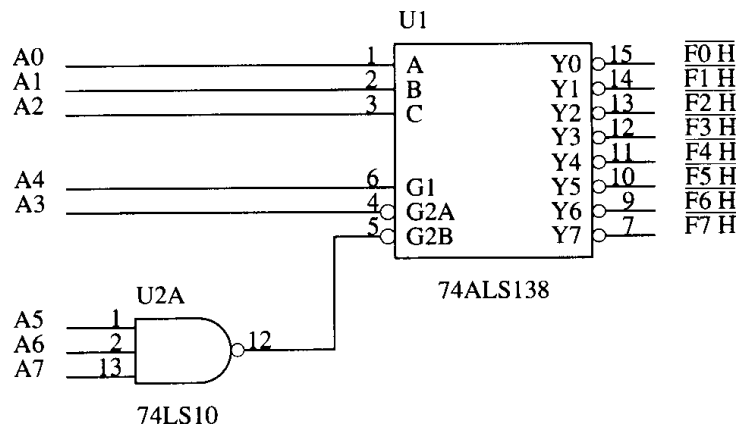
- Each address from F0H to F7H will be called as “port address”.
- Each port address will have eight (8) I/O lines (for each latch and buffer).

I/O Addresses:

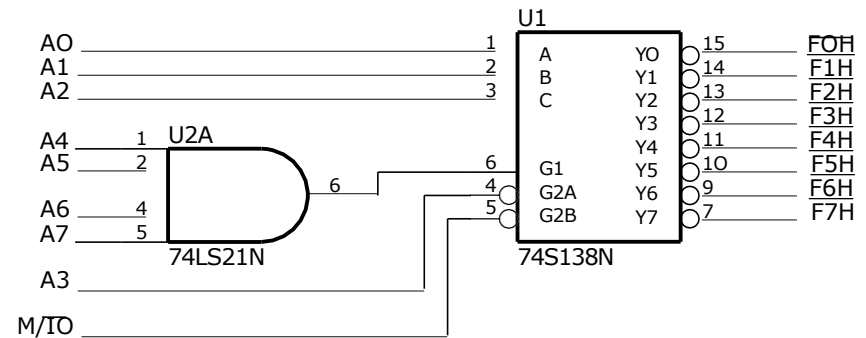
7	3	2	0
1111	0	000	
1111	0	001	
1111	0	010	
1111	0	011	
1111	0	100	
1111	0	101	
1111	0	110	
1111	0	111	

Since the total ports are eight (8) from F0H-F7H, we need to use a 3x8 decoder (**74LS138**) with $A_2 - A_0$ as the port select bits and $A_7 - A_3$ the decoder control bits.

I/O Address Decoder



A port decoder that decodes 8-bit I/O ports.
This decoder generates active low outputs for
ports F0H–F7H.



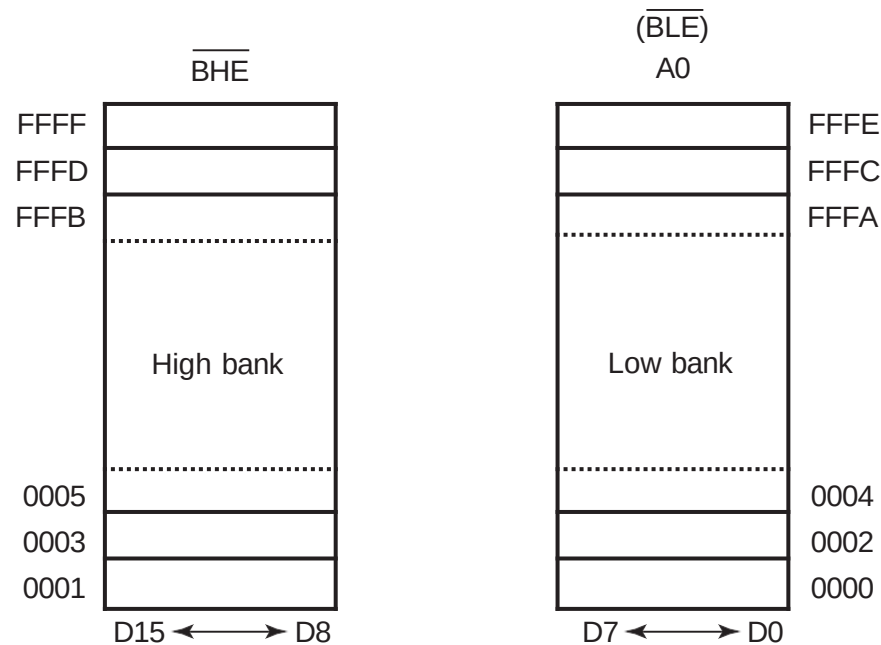
A proper decoder for 8-bit I/O with $\overline{M/I O}$ part of
the decoder control.

The output of the decoder will enable or control the I/O latches and buffers together with the $\overline{RD}$, $\overline{WR}$ and $\overline{M/I O}$ signals from the microprocessor.

8- and 16-bit Wide I/O Ports

- Data transferred to an 8-bit I/O device exist in one of the I/O banks in a 16-bit microprocessor such as the 8086.
- There are 64K different 8-bit ports, but only 32K different 8-bit ports because a 16-bit port uses two 8-bit ports.
- The I/O system on such a microprocessor contains two 8-bit memory banks, just as memory does.

8- and 16-bit Wide I/O Ports



The I/O banks found in the 8086, 80186, 80286, and 80386SX.

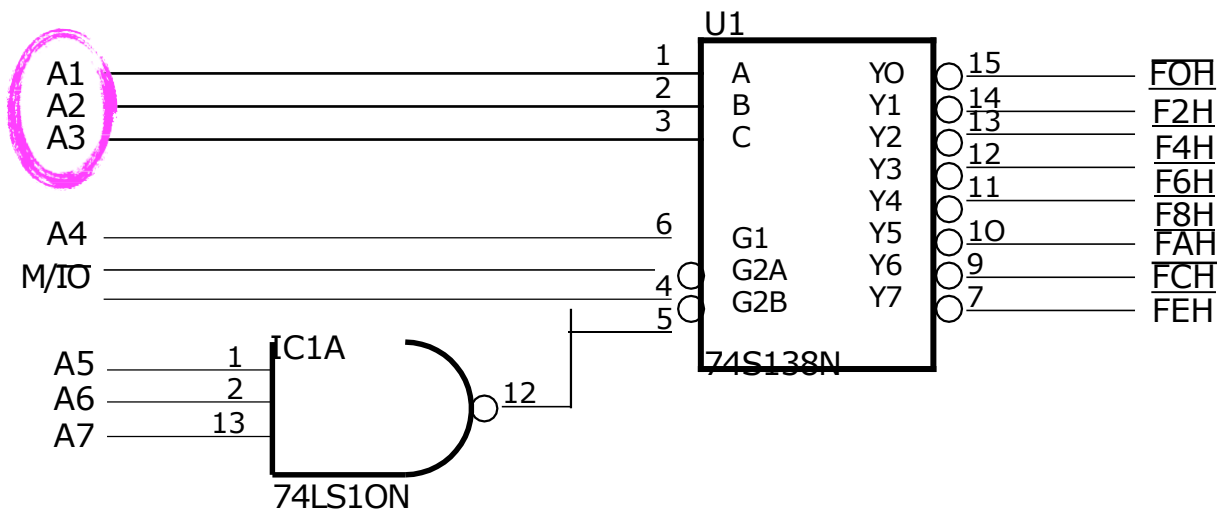
8086 and 8-bit I/O Ports

- Since the 8086 requires two banks (odd and even), accessing an 8-bit I/O port requires the use only of the **even (low)** bank addresses.

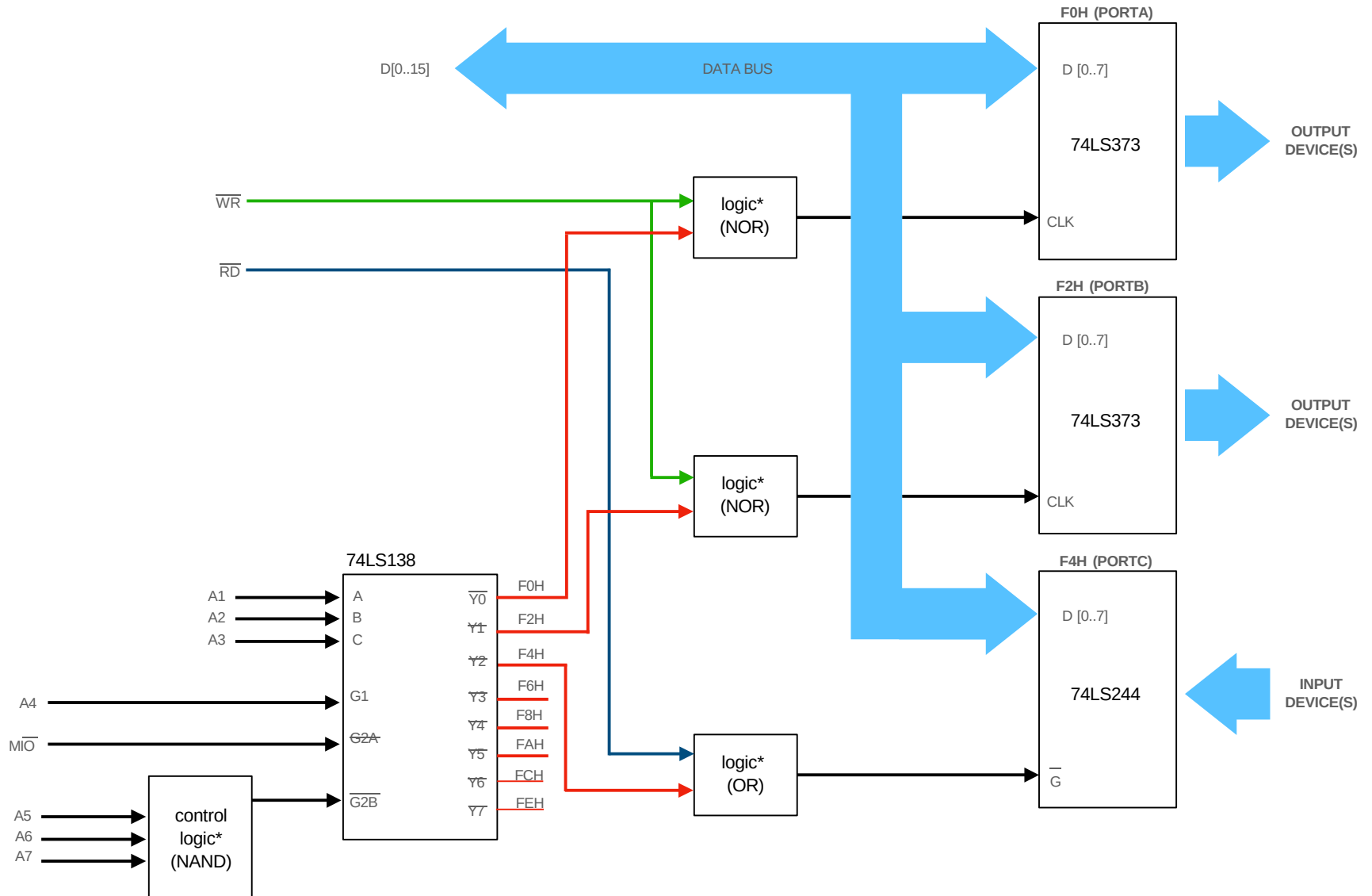
I/O Addresses:	7	3	1			
	1111	0000	✓	1111	1000	✓
	1111	0001		1111	1001	
	1111	0010	✓	1111	1010	✓
	1111	0011		1111	1011	
	1111	0100	✓	1111	1100	✓
	1111	0101		1111	1101	
	1111	0110	✓	1111	1110	✓
	1111	0111		1111	1111	

To decode only the even address, A_0 will **not** be part of the port select bits.
 Therefore, the address range will now be **F0H - FFH** to address eight (8) I/O ports.

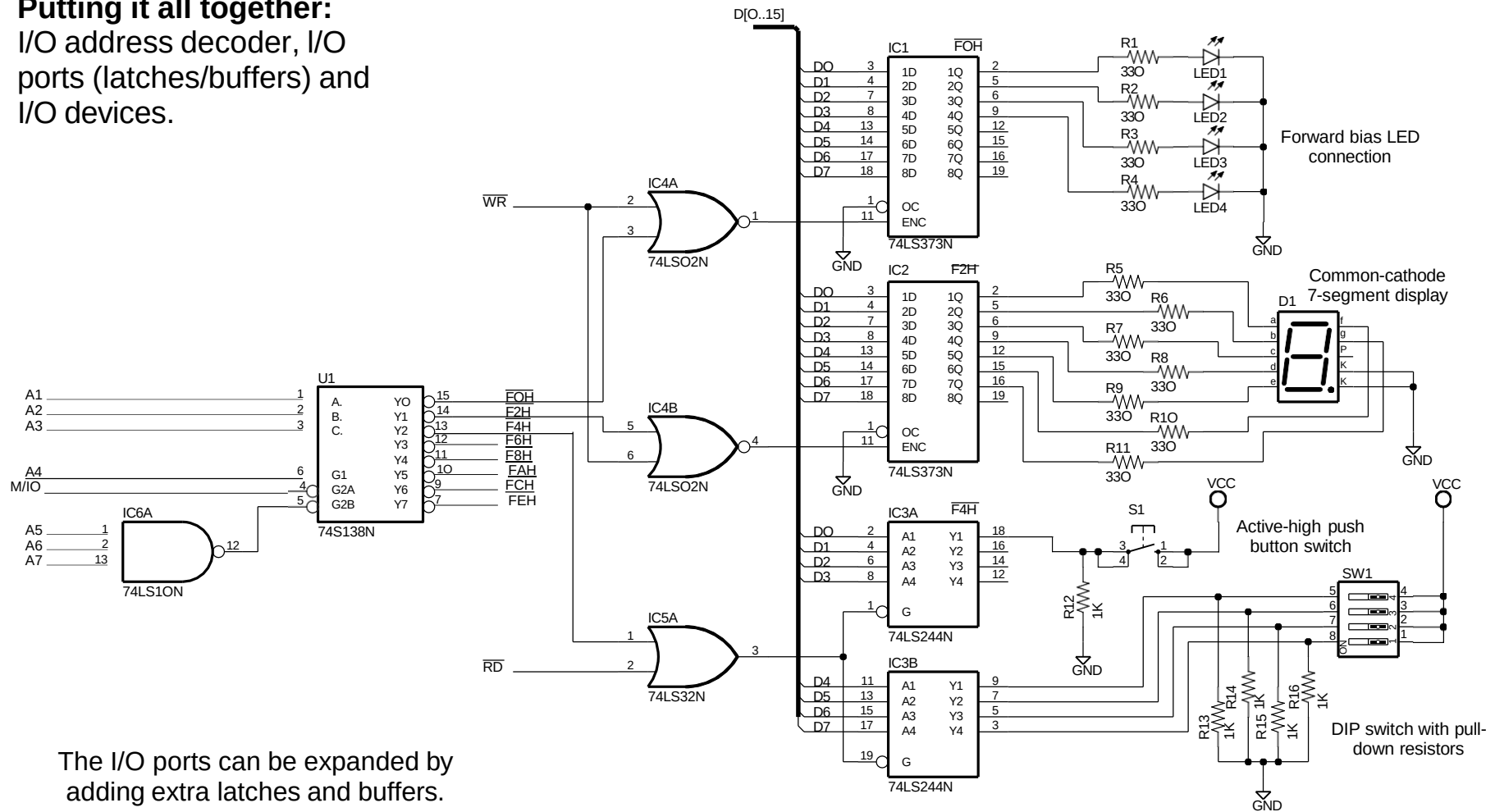
I/O Address Decoder



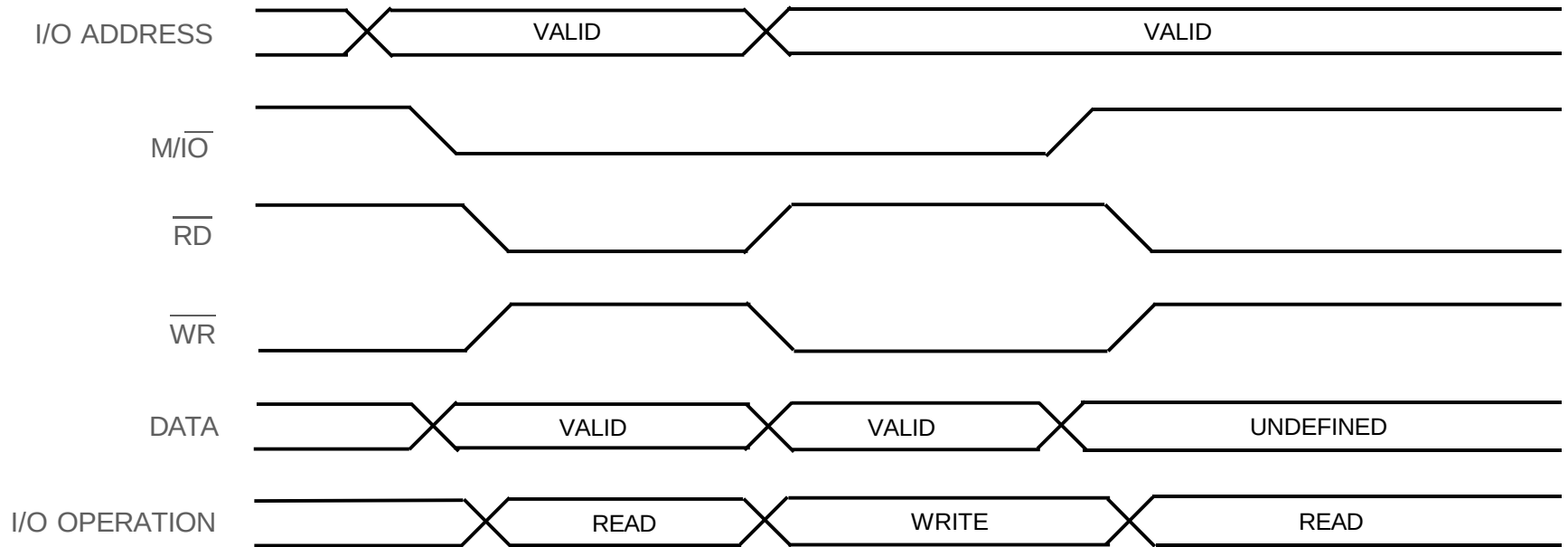
The decoder decodes only even addresses by not including A_0 as part of the port select bits.



Putting it all together:
I/O address decoder, I/O
ports (latches/buffers) and
I/O devices.



The I/O ports can be expanded by
adding extra latches and buffers.



I/O Timing Diagram

I/O Programming

- Writing and reading data to/from an I/O device require the use of IN, OUT, INS or OUTS instruction.
- Before performing read or write, the **port address** must be determined and set as the address.

I/O Programming

1. Based on the given circuit, the following program will turn on all the LEDs at port F0H:

```
MOV DX, 0F0H ; set the port address
MOV AL, 0FH  ; set data to be sent to I/O device
OUT DX, AL   ; send the data to the port
```

2. Read the push-button switch in port F2H:

```
MOV DX, 0F4H ; set the port address
IN AL, DX    ; read data from port
TEST AL, 01H ; check bit 0 if logic 1 (switch)
JZ LOGIC_1   ; jump to label LOGIC_1 if bit 0 of AL
              ; is not equal to 1
```

...

I/O Programming

3. Read the DIP switch data from port F4H:

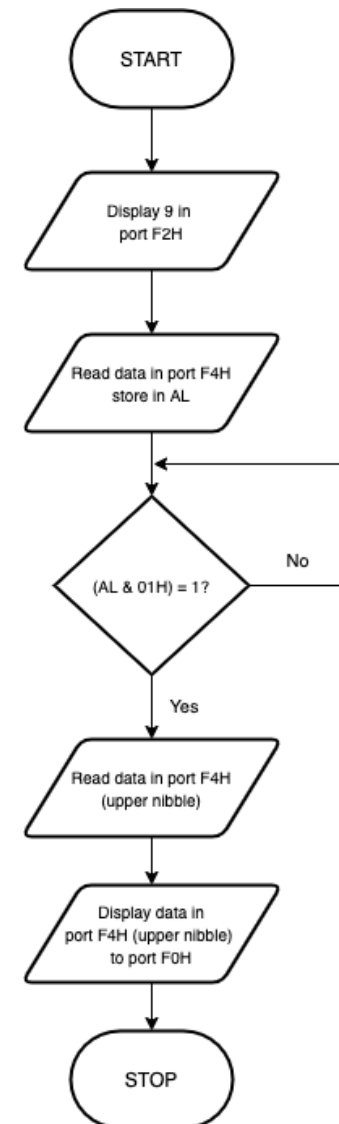
```
MOV DX, 0F4H ; set the port address
IN AL, DX    ; read data from port
MOV CL, 4
SHR AL, CL   ; move the data to the lower
              ; nibble position
              ; data will be read as 0XH
```

4. Display '9' in port F2H:

```
MOV DX, 0F2H ; set the port address
MOV AL, 01101111B ; set the segment to turn-on
OUT DX, AL    ; send the data to the port
```

Exercise

- Circuit on page 22 will be interfaced to the 8086 in Proteus.
- Number '9' will be displayed at port F2H.
- Port F0H will display the data according to input from the DIP switch in port F4H when S1 is pressed.

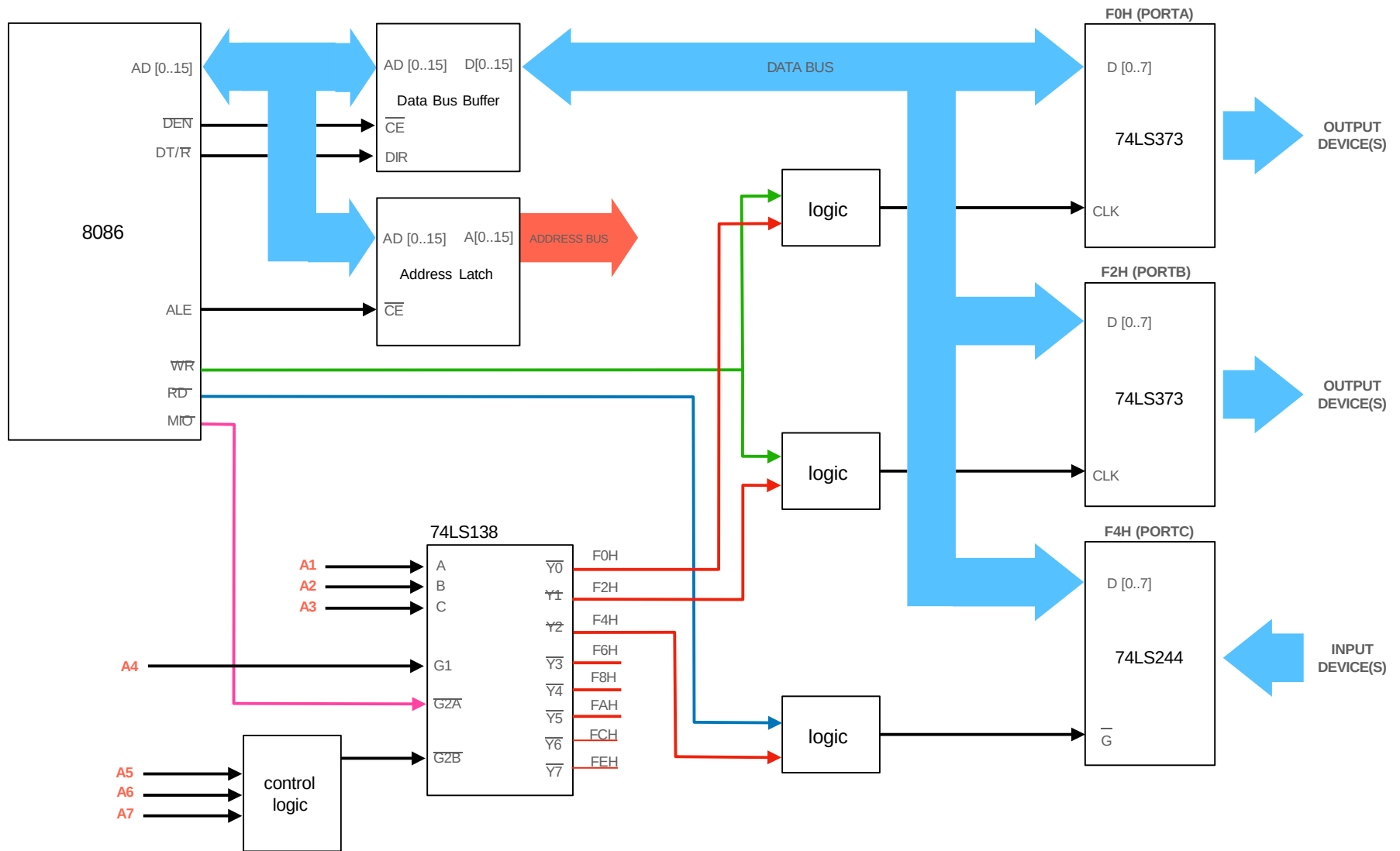


8086 VSM in Proteus

- The 8086 Virtual Simulation Model in Proteus does not support memory interfacing since both RAM and ROM are virtually embedded* in the 8086 itself.
- Therefore, in constructing a simulation circuit, only the address latch, data buffer, I/O address decoder and latches/buffer will be interfaced.
- In the VSM, the 8086 does not require an 8284** clock generator, we can simply edit the clock frequency on the device properties.

* The actual 8086 does not have a built-in RAM and ROM.

** In actual implementation, the 8086 does not have a built-in clock generator.



Interfacing basic I/O to 8086 in Proteus



```
DATA SEGMENT
    PORTA EQU 0F0H      ; address to access port F0H
    PORTB EQU 0F2H      ; address to access port F2H
    PORTC EQU 0F4H      ; address to access port F4H
DATA ENDS

CODE SEGMENT PUBLIC 'CODE'
    ASSUME CS:CODE
START:
    ; display '9' in 7-segment display
    MOV DX, PORTB        ; set port to F2H
    MOV AL, 01101111B    ; signal to display '9'
    OUT DX, AL           ; send data in AL to port F2H
THERE:
    ; check push-button switch
    MOV DX, PORTC        ; set port to F4H
    IN AL, DX            ; read data in port F4H
    TEST AL, 01H         ; check bit 0 if logic 1 (switch)
    JZ THERE             ; jump to THERE if bit 0 is not equal to 1
    ; read data from DIP switch
    MOV DX, PORTC        ; set port to F4H
    IN AL, DX            ; read data in port F4H
    MOV CL, 04H
    SHR AL, CL           ; move the data to the lower nibble
    ; display data from DIP switch to LEDs
    MOV DX, PORTA        ; set port to F0H
    OUT DX, AL           ; send data in AL to port F0H
    ; program enters endless loop*
HERE NOP                ; do nothing
    JMP HERE             ; jump to HERE
CODE ENDS
END START
```

* This is done to keep simulation running after code is executed.



University of San Carlos | Department of
COMPUTER ENGINEERING

CpE 3104 - Microprocessors

End of Lecture

Unit 6: INPUT/OUTPUT (I/O) INTERFACING

© 2025